

Lab4 面向对象编程：多态

学习目标：

1. 多态性是什么，它如何使程序更方便，以及它如何使系统更具可扩展性和可维护性。
2. 声明和使用 `virtual` 函数来实现多态性。
3. 抽象类和具体类之间的区别。
4. 创建抽象类。
5. C++如何实现 `virtual` 函数和动态绑定
6. 如何使用 `virtual` 析构函数来确保所有适当的析构函数都在一个对象上运行。

实验

实验一:(12.12)

1. 问题描述

（工资发放系统修改）修改图12.9-12.17中的工资发放系统，将私有数据成员 `birthDate` 包括在类 `Employee` 中。使用图10.6-10.7中的 `Date` 类来表示雇员的生日。假设每月处理一次工资单。创建一个 `Employee` 引用的 `vector` 对象，来存储各种雇员对象。在一个循环中（多态地）计算每个雇员的工资，如果当月是雇员的生日所在的月份，则在该雇员的工资总额中添加100美元奖金。

2. 解决问题小提示

如何获得系统时间

方法1:

```
#include <windows.h>

int main()
{
    SYSTEMTIME systm;
    GetLocalTime(&systm);

    cout<<systm.wYear<<"-"<<systm.wMonth<<"-"<<systm.wDay<<" "<<
systm.wHour<<":"<<systm.wMinute<<":"<<systm.wSecond;

    return 0;
}
```

方法2:

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    time_t nowtime;
    struct tm* ptm;
    time(&nowtime);
    ptm = localtime(&nowtime);
    cout<<ptm->tm_year + 1900<<"-"<<ptm->tm_mon + 1
    <<"-"<<ptm->tm_mday<<" "<<ptm->tm_hour<<":"
    <<ptm->tm_min<<":"<<ptm->tm_sec;
    return 0;
}
```

3. 输出如下

```
Employees processed polymorphically via dynamic binding:
salaried employee: John Smith
birthday: June 15, 1944
social security number: 111-11-1111
weekly salary: 800.00
earned $800.00

hourly employee: Karen Price
birthday: April 29, 1960
social security number: 222-22-2222
hourly wage: 16.75; hours worked: 40.00
HAPPY BIRTHDAY!
earned $770.00

commission employee: Sue Jones
birthday: September 8, 1954
social security number: 333-33-3333
gross sales: 10000.00; commission rate: 0.06
earned $600.00

base-salaried commission employee: Bob Lewis
birthday: March 2, 1965
social security number: 444-44-4444
gross sales: 5000.00; commission rate: 0.04; base salary: 300.00
earned $500.00

deleting object of class SalariedEmployee
deleting object of class HourlyEmployee
deleting object of class CommissionEmployee
```

实验二:

1. 问题描述

(形状层次结构)使用11.7中设计的形状层次结构。

每个TwoDimensionalShape都应该包含函数getArea来计算二维形状的面积。每个ThreeDimensionalShape都应该具有成员函数getArea和getVolume，以分别计算三维形状的表面积和体积。

创建一个程序，使用Shape指针的vector，存储指向层次结构中每个具体类的对象的指针。程序需要应该打印每个vector元素所指向的对象。此外，在处理vector中所有形状的循环中，确定每个形状是TwoDimensionalShape还是ThreeDimensionalShape。如果形状是二维形状，则显示其面积。如果形状是三维形状，则显示其面积和体积。

2.要求

- (1) 定义至少5个类
- (2) 为每个类定义成员函数，必须包括以下函数：构造函数、析构函数、virtual getArea函数
- (3) 主函数必须包括：声明vector，测试getArea函数。

实验三:(12.14)

(使用账户层次的多态银行程序)使用11.10中创建的Account类层次结构开发一个具有多态性的银行系统程序。创建一个Account指针的vector对象，其中的指针指向 SavingsAccount 和 CheckingAccount 对象。对于该vector中指针指向的每一个 Account，允许用户使用成员函数debit制定要从该Account取出的货币金额，并允许用户使用成员函数credit指定存入该Account的货币金额。处理每个Account时，应判定它的类型。如果Account是SavingsAccount，就使用calculateInterest计算该账户应该得到的利息，然后使用成员函数credit把利息加到账户余额。在处理完账户后，通过调用基类成员函数getBalance获得的更新后的账户余额并打印。

